



Schedulr.

Version-controlled specification schedules for architecture and interiors practices

Software Engineering Bootcamp · Capstone Project · 2026

Presented by Hannah Williams

The Problem!

1

Productivity - Nothing is remembered, Nothing is shared

That perfect product you specified in 2023. Or was it 2024? Six hours of folder-diving later... Ouch it was 2022. You find the perfect tile six months too late, because it was living in someone else's spreadsheet.

2

Document Control - Everything is retyped (or worse copied and pasted)

The same 30 products, typed again and again every project but with different typos.
Sent out a revision – accidentally deleted the tab

3

Document Integrity - or worse copied and pasted

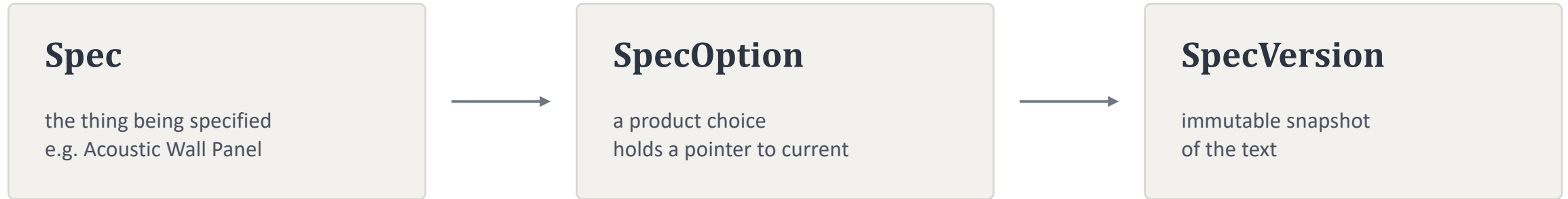
The copied schedule you just sent the client still cites the 2016 building code; the year of the project you copied it from.

I work in an architecture and interiors practice; believe me I have seen the problem! many times, with my own eyes.



The Solution!

An edit adds a version — it never overwrites one



What happens when someone edits a spec

- 1 The old version is left exactly as it is
- 2 A new **SpecVersion** is written, numbered 2
- 3 **SpecOption.currentVersionID** moves to point at it
- 4 Any read follows the pointer, so it lands on the new version

An edit adds a version — it never overwrites one

Spec

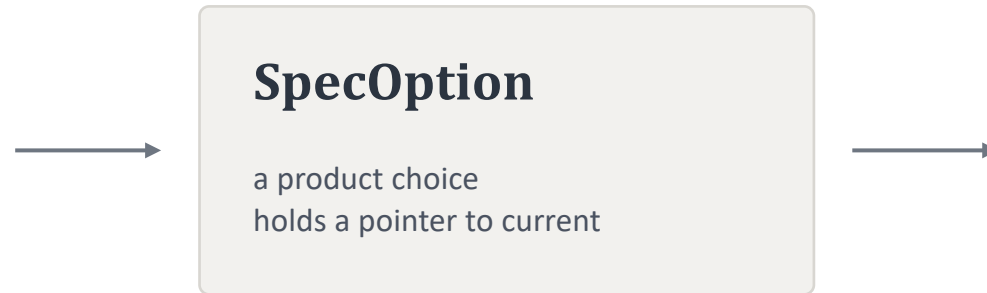
the thing being specified
e.g. Acoustic Wall Panel



backend/src/models/specModel.js

```
const SpecSchema = new Schema({
  {
    orgId: { type: Schema.Types.ObjectId, ref: "Organisation", required: true },
    ownerType: {
      type: String,
      enum: ["library", "schedule"],
      required: true,
      immutable: true, // set at birth, never changes
    },
    // ... code trimmed
    category: { type: String, trim: true },
    subCategory: { type: String, trim: true },
    // ... code trimmed
  },
  { timestamps: true },
});
```

An edit adds a version — it never overwrites one



```
const SpecOptionSchema = new Schema(  
  {  
    specID: { type: Schema.Types.ObjectId, ref: "Spec", required: true },  
    currentVersionID: { type: Schema.Types.ObjectId, ref: "SpecVersion" }, // null at creation  
    derivedFromVersionID: { type: Schema.Types.ObjectId, ref: "SpecVersion" }, // breadcrumb in  
    pushedAsOptionID: { type: Schema.Types.ObjectId, ref: "SpecOption" }, // breadcrumb out  
    isRedundant: { type: Boolean, default: false },  
    createdBy: { type: Schema.Types.ObjectId, ref: "User" },  
  },  
  { timestamps: true },  
);
```

backend/src/models/specOptModel.js

An edit adds a version — it never overwrites one



SpecVersion

immutable snapshot
of the text

```
const SpecVersionSchema = new Schema({
  optionID: { type: Schema.Types.ObjectId, ref: "SpecOption", required: true },
  supplierID: { type: Schema.Types.ObjectId, ref: "Supplier" },
  versionNumber: { type: Number, required: true },
  productName: { type: String, trim: true },
  rawText: { type: String, trim: true },
  imageKey: { type: String, trim: true },
  internalComments: { type: String, trim: true },
  attributes: [SpecVersionAttributeSchema],
  createdBy: { type: Schema.Types.ObjectId, ref: "User" },
  createdAt: { type: Date, default: Date.now },
});
```

backend/src/models/specVerModel.js

Making a web grid behave like a spreadsheet

1

Multi-line editing inside a cell

2

Images rendered in the row

Making a web grid behave like a spreadsheet

1

Multi-line editing inside a cell

A DataGrid commits the row when you press Enter. A spec needs Enter to mean a new line. A key guard intercepts it inside the Specification cell, so the text box behaves the way a designer expects and clicking away is what saves.

```
// The guard tells the grid to ignore Enter, but only in edit mode and
only in the named columns.
export const multilineEnterGuard =
  (fields) =>
    (params, event) => {
      if (
        params.cellMode === "edit" &&
        event.key === "Enter" &&
        fields.includes(params.field)
      ) {
        event.defaultMuiPrevented = true;
      }
    };
//frontend/src/data/taxonomy.jsx

onCellKeyDown={multilineEnterGuard(["spec"])}
// frontend/src/pages/CategoryLibrary.jsx
```

```
//The editor itself also stops Enter reaching the grid.
onKeyDown={(event) => {
  if (event.key === "Enter") event.stopPropagation();
}}//frontend/src/components/MultilineEditCell.jsx
```

Making a web grid behave like a spreadsheet

2

Images rendered in the row

Paste an image URL and it previews live in the edit cell, then renders full-width in the grid — so a finishes schedule can be read visually, the way a sample board is.

```
//Typing a URL previews it straight away in the editor.
{value ? (
  <img
    src={value}
    alt=""
    style={{ width: "100%", objectFit: "contain", display:
      "block" }}
    onError={(e) => (e.target.style.display = "none")}
    onLoad={(e) => (e.target.style.display = "block")}
  />
) : null} //^frontend/src/components/ImageEditCell.jsx
```

```
// And the saved row renders it full-width.
renderCell: ({ value }) =>
  value ? (
    <img
      src={value}
      alt=""
      style={{ width: "100%", height: "auto", objectFit:
        "contain" }}
    />
  ) : null, //frontend/src/pages/CategoryLibrary.jsx
```

One text box in, structured data out

What the designer types

```
Product: Echopanel 24mm
Colour: Onyx 550
Size: W1200 x L2800
NRC: Direct fix 0.65
Supplier: Woven Image
```

parse
→

What the database stores

```
rawText    the text, kept verbatim

attributes[]
  { key: "Colour", value: "Onyx 550" }
  { key: "Size",   value: "W1200 x L2800" }
  ... sortOrder preserved
```

Typing stays as fast as a spreadsheet cell, but the data comes out structured - ready for filtering, comparison and export later, no forms

```
//Raw text goes in attribute comes out
const colonIndex = trimmed.indexOf(":");
const possibleKey = colonIndex > 0 ? trimmed.slice(0, colonIndex).trim() : null;

const isNewAttribute =
  possibleKey &&
  possibleKey.length <= KEY_MAX_LENGTH &&
  !possibleKey.includes(",") &&
  /^[A-Za-z][A-Za-z0-9 \-/&()]*$/ .test(possibleKey);

if (isNewAttribute) {
  attributes.push({
    key: possibleKey,
    value: trimmed.slice(colonIndex + 1).trim(),
    sortOrder: attributes.length + 1,
  }); //backend/src/services/specTextParser.js
```

```
//Anything that isn't a new key carries on the last value
//This is what lets a spec run over several lines without breaking.
} else if (attributes.length > 0) {
  //continuation of the previous value
  attributes[attributes.length - 1].value += " " + trimmed;
} else {
  //text before any key appears - treat as an untitled description
  attributes.push({ key: "Description", value: trimmed, sortOrder: 1 });
} //backend/src/services/specTextParser.js
```

 **Schedulr.**

with **Spreadsheet ease.**

but **Nothing forgotten.**

and **Everything shared.**

Thank you — happy to take questions.

For more information, please refer to the [README](#) and [Project Documentation](#).